



Grado en ingeniería informática

Sistemas distribuidos 25/26

Grupo 81

Práctica 1

«Ejercicio evaluable 2»

Chen Xu — 100522395

Miguel Amiel Gil — 100525454

Profesor

Felix García Carballeira

1. Diseño y compilación

1.1. Diseño y elaboración del protocolo

En esta práctica debemos desarrollar un entorno cliente-servidor que se comunica mediante sockets. En consecuencia, la información se debe transmitir utilizando una serialización global e independiente del lenguaje de programación utilizado y de las características del sistema. En nuestro caso, hemos considerado el intercambio de mensajes mediante cadenas de texto.

1.1.1. Diseño

Para el desarrollo de la práctica hemos considerado el servidor elaborado en la práctica anterior pero modificando el método de comunicación por sockets en vez de colas de mensajes. El servidor posee un socket que recibe las peticiones de los clientes y genera un nuevo socket que añade al buffer global. A continuación, los threads de la pool van obteniendo estos descriptores de socket y gestionan la comunicación con ese cliente en concreto. Cada thread trata una única petición del cliente, le devuelve el resultado y cierra la comunicación con el mismo. Luego, obtienen otro descriptor del buffer y continúan tratando nuevas peticiones. Se ha seguido la misma lógica que en el laboratorio anterior, en el servidor, se han usado mutex y variables condiciones para gestionar la concurrencia y en `claves.c` `file_locks`. Además, se han elaborado dos nuevas funciones que permiten al servidor tratar con las peticiones: `obtener_params` y `enviar_datos_get`. `Obtener_params` recibe los parámetros necesarios para gestionar la petición y `enviar_datos_get` devuelve los datos recuperados al servidor cuando sea necesario.

1.1.2. Protocolo

La implementación de nuestro protocolo de comunicación varía ligeramente en el lado servidor y cliente. La información no se manda de forma conjunta si no que se envía de forma progresiva en forma de paquetes. El formato de los paquetes es el siguiente (se enviarán unos u otros en función de la instrucción deseada):

Key -> string de máximo 32 chars Value1 -> string de máximo 32 chars N_value2 -> número codificado como string (número entre 1 y 32) V_value2 -> array de floats codificado como string ej: «[1,2,3]» value3 -> struct paquete codificado como un array de ints [x,y,z] ej «[4,5,6]»

1. Servidor: el servidor siempre recibe primero el código de operación que indica la instrucción a realizar. Una vez obtenido este código se realiza la recepción de los parámetros dependiendo del código de operación recibido.

Recepción params en función de la instrucción:

- DESTROY -> No recibe params
- GET_VALUE -> Server espera recibir la key
- SET_VALUE -> server espera recibir la key y los valores asociados a la key
- EXIST -> Server espera recibir la key
- MODIFY_VALUE -> server espera recibir la key y los valores asociados a la key
- DELETE_KEY -> server espera recibir la key

Envío params en función de la instrucción:

- PARA TODAS LAS INSTRUCCIONES -> se envía el código de ejecución
 - PARA GET_VALUE -> también se envían los valores recuperados
2. Cliente: el cliente siempre envía el código de ejecución primero. Después procede a enviar los parámetros al servidor siguiendo las mismas indicaciones que el protocolo de recepción del servidor. Por otro lado, el cliente recibe los parámetros de la misma forma que el protocolo de emisión del servidor.

1.2. Compilación

Para realizar la compilación del programa se utiliza el comando «make». Para ejecutar el servidor se ha de abrir una terminal separada y correrlo usando «./servidor-sockets 3000». Para la correcta ejecución de los clientes asume el uso del puerto 3000 por el servidor. Para correr los clientes hemos incluido un archivo .sh llamado «run.sh» para ejecutar los tests de comprobación de la funcionalidad y «run_d.sh» para correr los tests de comprobación de la concurrencia. Dentro de estos se declaran las variables de entorno necesarias para la correcta ejecución de los clientes.

```
// Terminal 1
make
./servidor-sock 3000 # Hacer después de la línea uno de terminal 2
// Terminal 2
./run.sh # Ejecutamos los tests que comprueban la funcionalidad de las
instrucciones de claves.c
./run_d.sh # Ejecutamos los tests de la parte distribuida
```

1.3. Tests que comprueban funcionalidad base

ID	Test	Explicación	Entrada	Salida
1	Añadir clave (set_value)	Inserción correcta de una nueva clave inexistente.	key: «clave1» value1: «test1_value1» N_v2: 4 V_v2: {1.5, ...} value3 {1, 2, 3}	0
2	Recuperar la clave anterior	Obtener los resultados establecidos en el get anterior	key: «clave1»	0 value1: «test1_value1» N_v2: 4 V_v2: {1.5, ...} value3 {1, 2, 3}
3	Añadir clave existente	Error la clave ya existe	key: «clave1»	-1
4	Modificar la clave1	Modificación exitosa	key: «clave1» value1: «TEST4» N_v2: 4 V_v2: {4.1, ...} value3 {4.5, 4.6, 4.7}	0

5	Obtenemos la clave modificada en el test 4	Éxito, se recuperan los datos modificados	key: «clave1»	0 Valores recuperados key: «clave1» value1: «TEST4» N_v2: 4 V_v2: {4.1, ...} value3 {4.5, 4.6, 4.7}
6	Modificar clave no existente	Error la clave no existe	key: «no_existo»	-1
7	Modificamos clave no N_value2 fuera de rango	Error N_value2 está fuera de rango	key: «clave1» value1: «no_deberia_estar» N_v2: 33 V_v2: {9, ...}, value3 {0,56,456}	-1
8	get_value en clave no existente	Error porque la clave no existe	key: «no_existo» value1: «inexistencia» N_v2: 5 V_v2: {9, ...}, value3 {0, 56, 456}	-1
9	Comprobar existencia de clave1	Éxito la clave existe	key: «clave1»	1
10	Comprobar inexistencia de una clave	Error la clave no existe	key: «no_existo»	0
11	Borrar clave (delete_key), borramos la clave1	Eliminación correcta de una clave existente.	key: «clave11»	0
12	Borrar clave (delete_key), intentamos borrar de nuevo la clave1	Error al intentar borrar una clave que ya fue borrada.	key: «clave1»	-1
13	Creamos una serie de keys y las borramos con el destroy	Eliminación de todas las keys creadas.	No hay argumentos	0

1.4. Resultados de los tests de análisis de la concurrencia

Para analizar la concurrencia del servidor hemos elaborado tres archivos.c que realizan instrucciones sobre la misma clave. Concretamente, **app-cliente-4.c** borra la

clave «clave_compartida» periódicamente; **app-cliente-2** que intenta crear la clave «clave_compartida» y modificar sus datos de forma periódica y **app-cliente-3.c** que recupera los datos de la clave «clave_compartida».

En la siguiente tabla se puede observar como en función del orden de ejecución de cada de los archivos .c descritos, los resultados varían. El orden de creación, borrado y modificación de la key «clave_compartida» cambia.

Fichero	Explicación del test	Resultado obtenido
app-cliente-4	En este test de muestra una ejecución del archivapp-cliente-4.c. Este archivo borra continuamente la clave «clave_compartida». El orden de ejecución de las instrucciones influncia los resultados de las operaciones de cliente-lector y escritor. Cuando el destructor borra con éxito una clave, si el cliente-escriptor intenta modificar la clave o el lector acceder a la misma, estos obtienen un error.	[CAOS] Iniciando test de concurrencia... [ERROR] No se pudo borrar la clave compartida! [OK] Clave compartida borrada! [ERROR] No se pudo borrar la clave compartida! [ERROR] No se pudo borrar la clave compartida! [ERROR] No se pudo borrar la clave compartida! [ERROR] No se pudo borrar la clave compartida! [ERROR] No se pudo borrar la clave compartida! [CAOS] ¡Terminado!
app-cliente-2	En este test de muestra una ejecución del archivo app-cliente-2.c. Este archivo crea y modifica continuamente la clave «clave_compartida». El orden de ejecución de las instrucciones influncia los resultados de las operaciones de cliente-destructor y lector. Si el cliente modifica una key con éxito, el lector podrá observar dicho cambio. Si no se pudo modificar la key, se imprime un error. Este caso de error solamente ocurre si el cliente destructor ha borrado la clave con anterioridad.	[ESCRITOR] Iniciando test de concurrencia... [ESCRITOR] SET: value10 [ERROR] La key no existe: clave_compartida no se pudo escribir el nuevo valor1: value11 [ESCRITOR] SET: value12 [ERROR] La key no existe: clave_compartida no se pudo escribir el nuevo valor1: value13 [ESCRITOR] SET: value14 [ESCRITOR] MODIFY: value15 [ESCRITOR] ¡Terminado!
app-cliente-3	En este test se muestra una ejecución del archivo app-cliente-3 Este archivo intenta acceder continuamente a la clave «clave_compartida». En función del resto de clientes, se obtiene un valor1 distinto. Si no existe la key, imprime un error.	[LECTOR] Iniciando test de concurrencia... [LECTOR] ERROR al leer la clave. [LECTOR] Leído con éxito con valor1: value10 [LECTOR] ERROR al leer la clave. [LECTOR] ERROR al leer la clave. [LECTOR] ERROR al leer la clave. [LECTOR] ERROR al leer la clave. [LECTOR] ¡Terminado!